CS2201 Computer Organization & Architecture

L5: Division of Binary Numbers



John Jose

Associate Professor

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Binary Division

Dividend	:	7
Divisor	: ÷	3
Quotient	:	2
Remainder	:	1

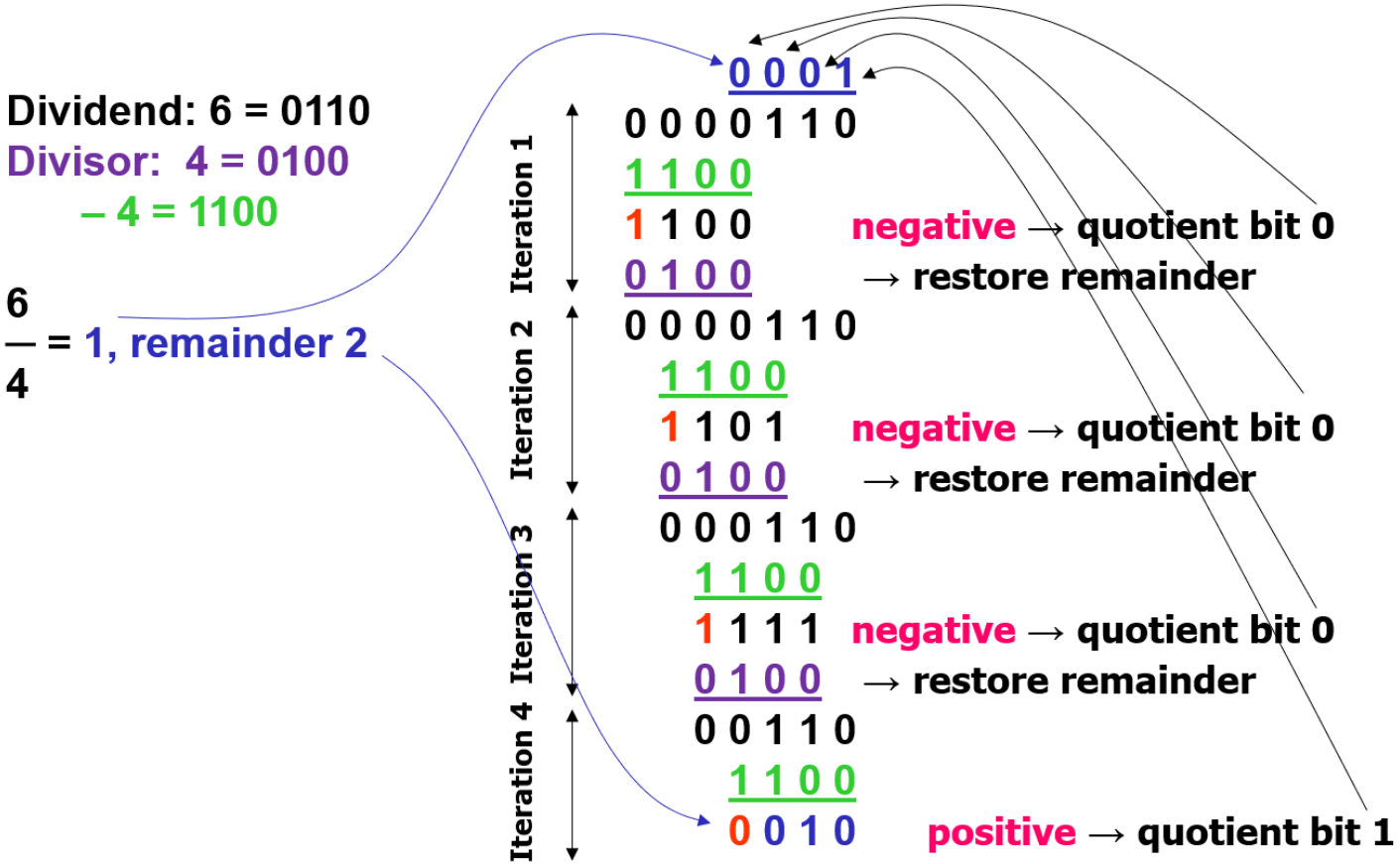
- Division is done by repeatedly subtracting the divisor from the dividend.
- Counting the number of times that the divisor can be subtracted from the dividend before the dividend becomes smaller than the divisor.
- Example: Division of 7 by 3
- Subtract repeatedly from 7, getting 4, 1, as intermediate results
- The quotient, 2, is the number of subtractions that had to be performed before the intermediate result became less than the dividend

Binary Division (Unsigned)

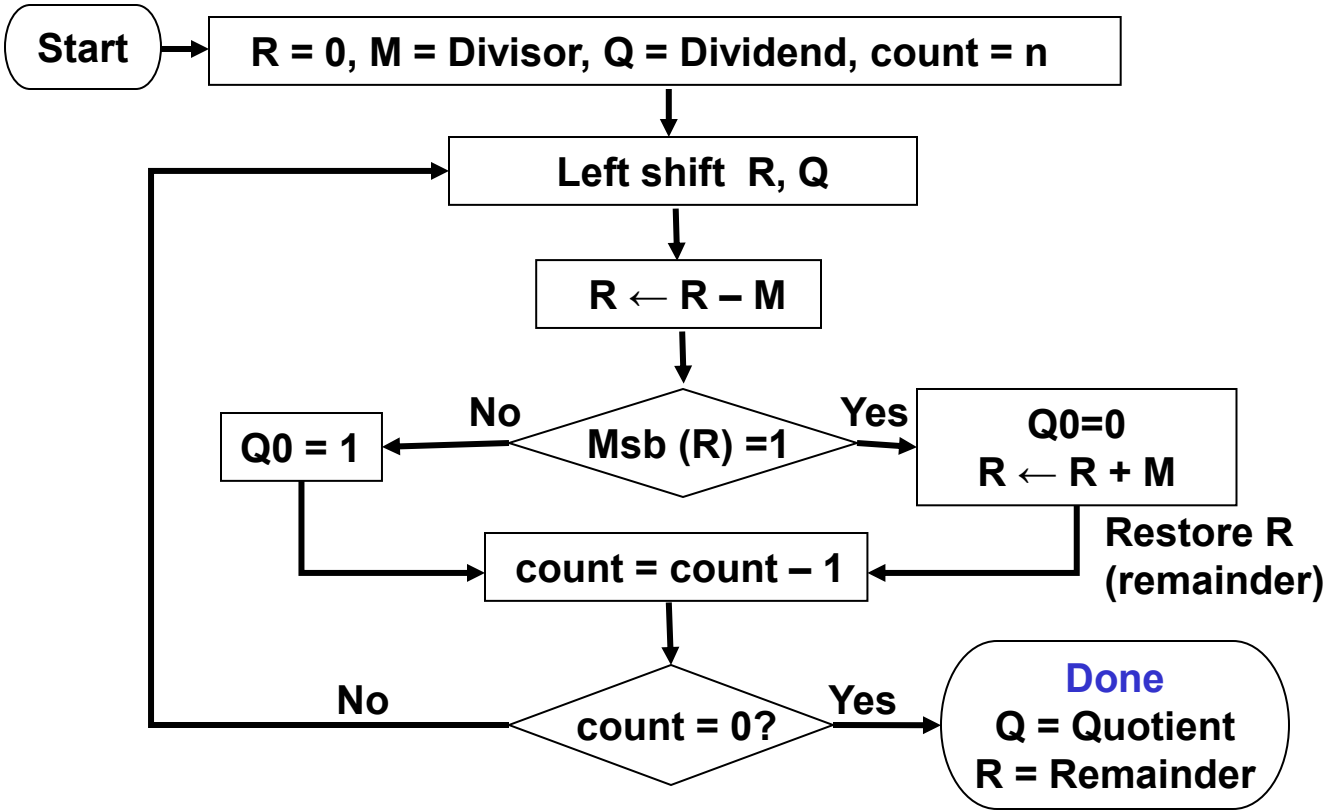
```
   1 3          Quotient
1 1 / 1 4 7     Divisor / Dividend
  1 1
    3 7
    3 3
      4         Remainder
```

```
     0 0 0 0 1 1 0 1
1 0 1 1 / 1 0 0 1 0 0 1 1
        1 0 1 1
      0 0 1 1 1 0
          1 0 1 1
        0 0 1 1 1 1
              1 0 1 1
                1 0 0
```

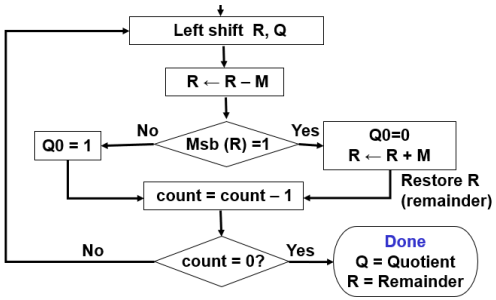
4-bit Binary Division (Unsigned)



Restoring Division Operation



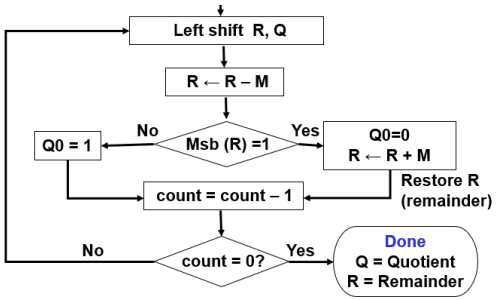
4-bit Example: 6/4 = 1, Remainder = 2



count	Actions	n	R, Q	M = Divisor
4	Initialize	4	0 0 0 0 0 | 0 1 1 0	0 0 1 0 0
	Shift left R, Q	4	0 0 0 0 0 | 1 1 0 *	0 0 1 0 0
	Add – M (11100) to R	4	1 1 1 0 0 | 1 1 0 *	0 0 1 0 0
3	Restore, add M (00100) to R	3	0 0 0 0 0 | 1 1 0 0	0 0 1 0 0
	Shift left R, Q	3	0 0 0 0 1 | 1 0 0 *	0 0 1 0 0
	Add – M (11100) to R	3	1 1 1 0 1 | 1 0 0 *	0 0 1 0 0
2	Restore, add M (00100) to R	2	0 0 0 0 1 | 1 0 0 0	0 0 1 0 0
	Shift left R, Q	2	0 0 0 1 1 | 0 0 0 *	0 0 1 0 0
	Add – M (11100) to R	2	1 1 1 1 1 | 0 0 0 *	0 0 1 0 0
1	Restore, add M (00100) to R	1	0 0 0 1 1 | 0 0 0 0	0 0 1 0 0
	Shift left R, Q	1	0 0 1 1 0 | 0 0 0 *	0 0 1 0 0
	Add – M (11100) to R	1	0 0 0 1 0 | 0 0 0 *	0 0 1 0 0
0	Set LSB(Q) = 1	0	0 0 0 1 0 | 0 0 0 1	0 0 1 0 0

Remainder | Quotient

4-bit Example: 8/3 = 2, Remainder = 2



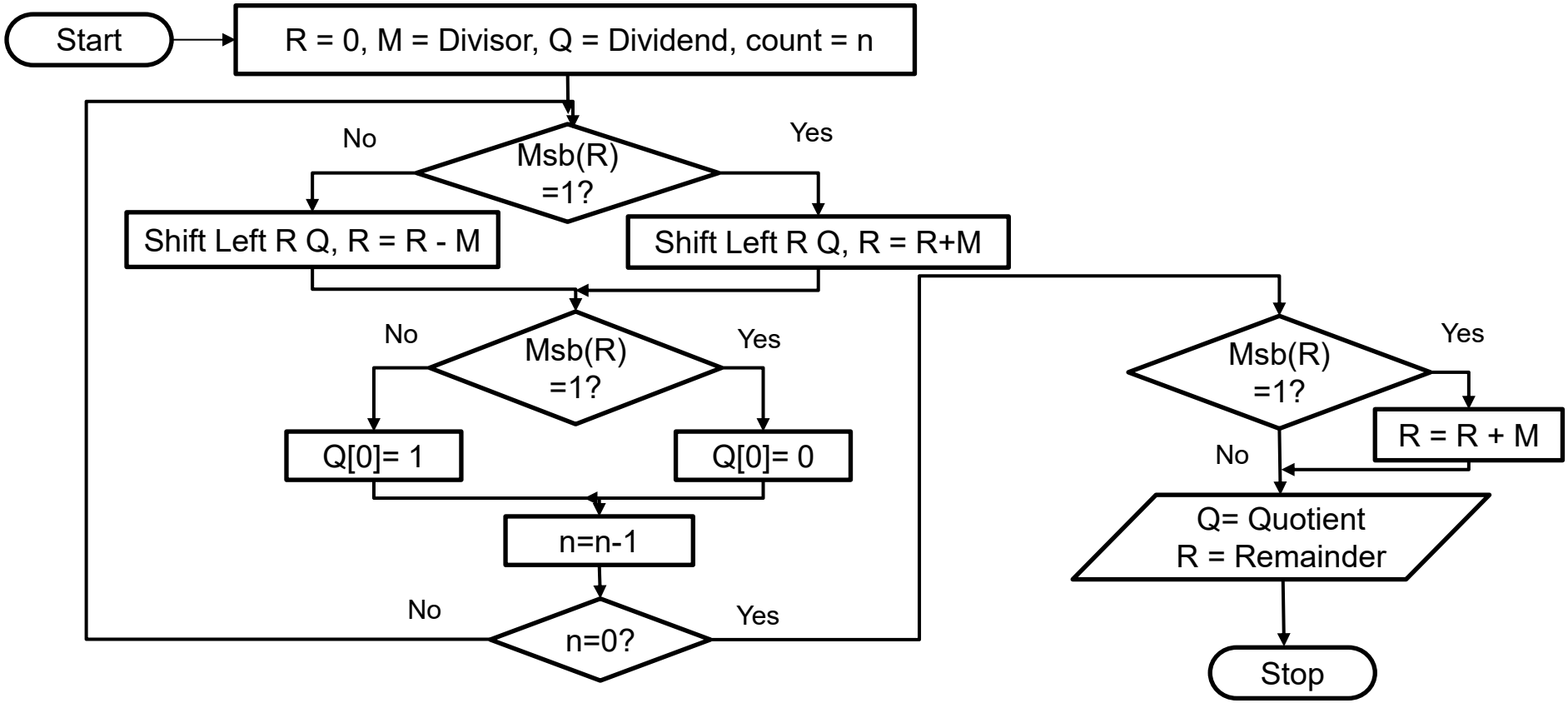
count	Actions	n	R, Q	M = Divisor
4	Initialize	4	0 0 0 0 0 | 1 0 0 0	0 0 0 1 1
	Shift left R, Q	4	0 0 0 0 1 | 0 0 0 *	0 0 0 1 1
	Add – M (11101) to R	4	1 1 1 1 0 | 0 0 0 *	0 0 0 1 1
3	Restore, add M (00011) to R	3	0 0 0 0 1 | 0 0 0 0	0 0 0 1 1
	Shift left R, Q	3	0 0 0 1 0 | 0 0 0 *	0 0 0 1 1
	Add – M (11101) to R	3	1 1 1 1 1 | 0 0 0 *	0 0 0 1 1
2	Restore, add M (00011) to R	2	0 0 0 1 0 | 0 0 0 0	0 0 0 1 1
	Shift left R, Q	2	0 0 1 0 0 | 0 0 0 *	0 0 0 1 1
	Add – M (11101) to R	2	0 0 0 0 1 | 0 0 0 *	0 0 0 1 1
1	Set LSB(Q) = 1	1	0 0 0 0 1 | 0 0 0 1	0 0 0 1 1
	Shift left R, Q	1	0 0 0 1 0 | 0 0 1 *	0 0 0 1 1
	Add – M (11101) to R	1	1 1 1 1 1 | 0 0 1 *	0 0 0 1 1
0	Restore, add M (00011) to R	0	0 0 0 1 0 | 0 0 1 0	0 0 0 1 1

Remainder | Quotient

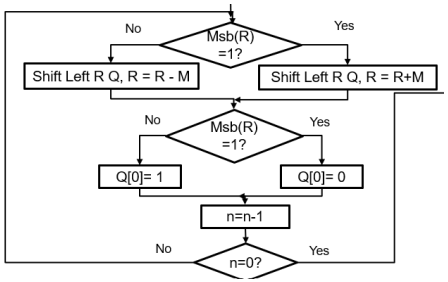
Non-Restoring Division

- Non-restoring division
  - Initialize and start iterations same as in restoring division by subtracting divisor
  - In any iteration after left shift and subtraction/addition
    - If R is positive, subtract divisor (M) in next iteration
    - If R is negative, add divisor (M) in next iteration
    - After final iteration, if R is negative then restore it by adding divisor (M)

Non-Restoring Division



4-bit Example: 8/3 = 2, Remainder = 2



count	Actions	n	R, Q	M = Divisor
4	Initialize	4	0 0 0 0 0 | 1 0 0 0	0 0 0 1 1
	Msb(R)=0, Shift left R, Q	4	0 0 0 0 1 | 0 0 0 *	0 0 0 1 1
	Add – M (11101) to R	4	1 1 1 1 0 | 0 0 0 *	0 0 0 1 1
3	Set LSB(Q) = 0	3	1 1 1 1 0 | 0 0 0 0	0 0 0 1 1
	Msb(R)=1, Shift left R, Q	3	1 1 1 0 0 | 0 0 0 *	0 0 0 1 1
	Add M (00011) to R	3	1 1 1 1 1 | 0 0 0 *	0 0 0 1 1
2	Set LSB(Q) = 0	2	1 1 1 1 1 | 0 0 0 0	0 0 0 1 1
	Msb(R)=1, Shift left R, Q	2	1 1 1 1 0 | 0 0 0 *	0 0 0 1 1
	Add M (00011) to R	2	0 0 0 0 1 | 0 0 0 *	0 0 0 1 1
1	Set LSB(Q) = 1	1	0 0 0 0 1 | 0 0 0 1	0 0 0 1 1
	Msb(R)=0, Shift left R, Q	1	0 0 0 1 0 | 0 0 1 *	0 0 0 1 1
	Add – M (11101) to R	1	1 1 1 1 1 | 0 0 1 *	0 0 0 1 1
0	Set LSB(Q) = 0	0	1 1 1 1 1 | 0 0 1 0	0 0 0 1 1
	Msb(R)=0, Add M (00011) to R	0	0 0 0 1 0 | 0 0 1 0	0 0 0 1 1

Restoring Vs Non-Restoring Division

- The fundamental logic difference lies in how the hardware reacts to an incorrect quotient bit guess (when a subtraction results in a negative partial remainder).
- **Restoring Division uses Immediate Rollback Logic:**
  It treats a negative remainder as an error that must be undone immediately. The hardware pauses, cancels the bad subtraction by adding the divisor back, and only then moves to the next bit.
- **Non-Restoring Division uses Look-Ahead Algebraic Logic:**
  It treats a negative remainder as a valid intermediate state. It allows the remainder to stay negative, shifts it, and mathematically corrects the error in the next cycle by performing an addition instead of a subtraction.

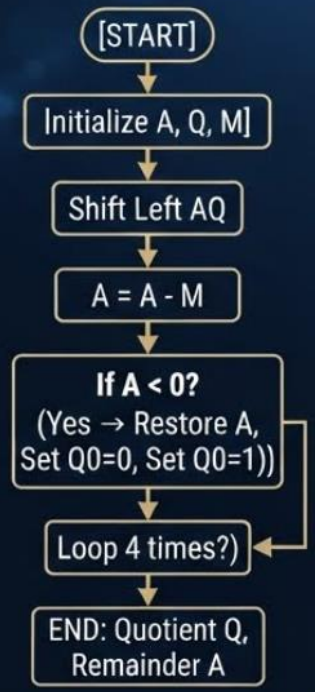
Restoring Vs Non-Restoring Division

- If a subtraction yields a negative remainder (R), the restoring method executes two operations to prepare for the next loop:
  - Restore: R + M
  - Shift Left: 2 × (R + M) = 2R + 2M
  - Next Trial Subtraction: (2R + 2M) - M = 2R + M
- Non-Restoring division skips immediate restoration. It simply shifts the negative remainder (2R) and immediately adds the divisor (M) in the next cycle → 2R+M
- Both yield the exact same mathematical expression (2R + M).

14/3 Using Restoring Division Method

Dividend (Q): 1110 (14) | Divisor (M): 00011 (3) | Control: n = 4 cycles

CYCLE	STEP ACTION	A (Accumulator)	Q (Quotient)	STATUS & BIT DECISION	
Initial	Start State	0 0 0 0 0	1 1 1 0	n = 4	
Cycle 1	AQ Shift Left AQ	0 0 0 0 1	1 1 0 _	Negative	Q0=0
	Subtract M	1 1 1 1 0 Negative	1 1 0 0		
Cycle 2	AQ Shift Left AQ	0 0 0 1 1	1 0 0 _		Q0=1
	Subtract M	0 0 0 0 0 Zero	1 0 0 1		
Cycle 3	AQ Shift Left AQ	0 0 0 0 1	0 0 1 _		Q0=0
	Subtract M	1 1 1 1 0 Negative	0 0 1 0		
Cycle 4	AQ Shift Left AQ	0 0 0 1 0	0 1 0 _		Q0=0
	Subtract M	1 1 1 1 1 Negative	0 1 0 0		

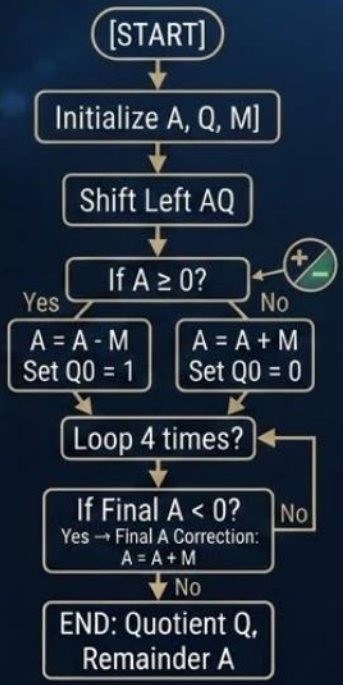


FINAL RESULT: Quotient (Q) = 0100_2 (4_{10}), Remainder (A) = 00010_2 (2_{10})

14/3 Using Non-Restoring Division Method

Dividend (Q): 1110 (14) | Divisor (M): 00011 (3) | Control: n = 4 cycles

CYCLE	STEP ACTION	A (Accumulator)	Q (Quotient)	STATUS & BIT DECISION	
Initial	Start State	0 0 0 0 0	1 1 1 0	n = 4	
Cycle 1	AQ Shift Left AQ A3 − Subtract M	0 0 0 0 1 − 1 1 1 1 0 Negative	1 1 0 _ 1 1 0 0	Negative	Q0=0
Cycle 2	AQ Shift Left AQ A3 + Add M	1 1 1 0 0 + 1 1 1 1 1 (-1)	1 0 0 0 1 0 0 1	Negative	Q0=0
Cycle 3	AQ Shift Left AQ − + Add M	1 1 1 1 0 + 0 0 0 0 1 Positive	0 0 0 1 0 0 1 0	Positive	Q0=1
Cycle 4	AQ Shift Left AQ − Subtract M	0 0 0 1 0 − 1 1 1 1 1 Negative	0 1 0 _ 0 1 0 0	*Final A is corrected because it is negative: A = A + M = 11111 + 00011 = 00010	Q0=0



FINAL RESULT: Quotient (Q) = 0100_2 (4_{10}), Remainder (A) = 00010_2 (2_{10})



johnjose@iitg.ac.in

http://www.iitg.ac.in/johnjose/

https://www.linkedin.com/in/john-jose-bb8431106/